

UNIDADE 3

**FUNDAMENTOS DA PROGRAMACIÓN
ORIENTADA A OBXECTOS**

v1.1

PROGRAMACIÓN

CICLO SUPERIOR DE DAM

Manuel Pacior Pérez



Índice

1 INTRODUCCIÓN.....	4
2 LIBERACIÓN DE MEMORIA EN JAVA.....	4
3 ESTRUCTURA E MEMBROS DA CLASE.....	4
4 TIPOS DE ATRIBUTOS.....	5
4.1 Atributos de Instancia.....	6
4.2 Atributos Estáticos.....	6
4.3 Atributos Finais.....	6
5 PAQUETES EN JAVA.....	7
5.1 Funcións principais dos paquetes.....	7
5.2 Paquetes estándar.....	7
5.3 Importar paquetes.....	8
5.4 Crear paquetes.....	8
5.5 Estructura de directorios.....	8
5.6 Paquetes aniñados (Sub-paquetes).....	8
6 MODIFICADORES DE ACCESO.....	9
6.1 default (sen modificador).....	9
6.2 public (acceso total).....	9
6.3 protected (acceso limitado).....	9
6.4 private (acceso restrinxido).....	9
7 MÉTODOS.....	10
7.1 Estructura dun método.....	10
7.1.1 Modificadores (opcional).....	10
7.1.2 Tipo de devolución.....	10
7.1.3 Nome.....	11
7.1.4 Parámetros (opcional).....	11
7.1.5 Corpo.....	11
7.1.6 Valor de Devolución (opcional).....	12
7.2 Paso de parámetros: primitivos vs Obxectos.....	12
7.2.1 Paso de parámetros con tipos primitivos.....	13
7.2.2 Paso de parámetros con obxectos.....	13
7.3 Construtores.....	16
7.4 Métodos de acceso: Getters e Setters.....	17



8 INSTANCIAR OBJECTOS.....	18
9 COMPARAR OBJECTOS.....	19
10 CLASES WRAPPER.....	21
10.1 Diferenzas cos tipos primitivos.....	22
10.2 Autoboxing e Unboxing.....	23
10.3 Principais métodos.....	23
10.4 Diferenzas ao comparar tipos primitivos e wrappers.....	24
11 CLASES DESTACADAS EN JAVA.....	24
11.1 Superclase Object.....	25
11.2 Clase Math.....	25
11.3 Clase Random.....	26
11.3.1 Características principais.....	26
11.3.2 Métodos para xerar números por tipo.....	26
11.3.3 Outras consideracións.....	27
11.3.4 Exemplos.....	27
11.3.4.1 Simulación dun dado de 6 caras.....	27
11.3.4.2 Xerar un número real entre dous límites.....	28
11.3.4.3 Xerar un número real entre dous límites.....	28
11.3.4.4 Simulación dun sorteo aleatorio.....	28
11.4 Clase String.....	28
11.4.1 Características principais.....	29
11.4.2 Principais métodos.....	29
12 EXEMPLOS.....	31
12.1 Xogo de adiviña a palabra.....	31
12.2 Clase Coche.....	33



1 INTRODUCCIÓN

Nesta unidade imos ver máis en detalle todos os elementos da POO na que nos iniciamos na unidade 2, para aprender a modelar pequenas aplicacións onde se traballe integrando os coñecementos vistos ao longo do curso no resto da unidades.

A continuación, imos facer un pequeno resumo dos diferentes elementos da POO cos que traballaremos nesta unidade, co obxectivo de implementar pequenas aplicacións en Java usando este paradigma de programación.

2 LIBERACIÓN DE MEMORIA EN JAVA

En Java, os obxectos créanse de maneira dinámica na memoria heap, e a súa destrución é xestionada automaticamente polo Garbage Collector (GC). Isto significa que, como programador, non tes que xestionar directamente a liberación da memoria, como ocorre en linguaxes como C ou C++. A continuación describimos o funcionamento deste mecanismo:

- 1) **Referencia aos Obxectos:** Mentres haxa unha referencia a un obxecto en uso, ese obxecto non será eliminado polo colector de lixo. Un obxecto é considerado "obsoleto" ou "inaccesible" cando non existen máis referencias que apunten a el, e nese momento é candidato a ser recollido polo GC.
- 2) **Proceso de Recollida:** O colector de lixo en Java segue diferentes algoritmos para detectar e eliminar obxectos non referenciados:
 - **Mark-and-Sweep:** O colector "marca" os obxectos que están en uso e despois "varre" os que non están marcados, para proceder a eliminalos.
 - **Compactación:** Logo de eliminar os obxectos non referenciados, o colector pode compactar a memoria para mellorar a súa eficiencia e dispoñibilidade.

Como vemos, Java libera a memoria automaticamente para os obxectos cando xa non se necesitan. Isto permite evitar problemas de fugas de memoria que son comúns noutros paradigmas onde a xestión de memoria é manual.

3 ESTRUCTURA E MEMBROS DA CLASE

Unha clase en Java é un modelo para crear obxectos, de xeito que cada obxecto é unha instancia



da clase e posúe as súas propias variables de instancia e métodos.

A estrutura básica é a seguinte:

```
public class NomeDaClase {  
    // Variables de instancia (atributos)  
    tipoDeDato nomeDaVariable;  
  
    // Constructor  
    public NomeDaClase(parámetros) {  
        // Inicialización de variables  
    }  
  
    // Métodos  
    tipoDeRetorno nomeDoMetodo(parámetros) {  
        // Corpo do método  
    }  
}
```

Os membros máis destacado dunha Clase son:

Variables de Instancia (atributos):

- Son variables que pertencen a cada instancia da clase.
- Pódense acceder e modificar en calquera método da clase.

Construtor:

- É un método especial que se executa cando se crea un novo obxecto da clase.
- Utilízase para inicializar as variables de instancia dun obxecto e cada obxecto ten a súa copia desas variables.

Métodos:

- Son funcións que definen o comportamento dun obxecto.
- Pódense chamar para facer operacións sobre as variables de instancia

4 TIPOS DE ATRIBUTOS

En Java, hai diferentes tipos de atributos que se poden definir dentro dunha clase e poden ser:

- Estáticos



- De instancia
- Finais

Cada un ten distintas características e usos propios, e deseguido imos ver unha descrición detallada de cada tipo.

4.1 Atributos de Instancia

- ✓ Son variables que se asocian con cada instancia dunha clase.
- ✓ Cada obxecto creado a partir da clase ten a súa propia copia destes atributos.
- ✓ Os obxectos poden ter métodos que alteran o seus datos.

Exemplo:

```
/** Km percorridos polo coche. */  
double km = 0;
```

4.2 Atributos Estáticos

- ✓ Son variables que se asociarán coa clase en si en lugar de coas instancias individuais.
- ✓ Todas as instancias da clase comparten a mesma copia destes atributos.
- ✓ Pódense acceder sen crear un obxecto específico da clase (sempre que teñamos acceso).

Exemplo:

```
/** Contador de coches. */  
static int contadorDeCoches;
```

4.3 Atributos Finais

- ✓ Son variables cuxos valores non poden ser modificados despois de ser inicializados.
- ✓ Pódense inicializar no momento da declaración ou no construtor da clase.
- ✓ O seu valor debe ser asignado unha vez e non pode ser cambiado despois diso.

Exemplo:

```
/** Marca do coche. Atributo final */  
final String marca;  
  
/** Ano de fabricación do coche. Atributo final */  
final int anoFabricacion;  
  
/** Prezo do coche. Atributo final */
```



```
final double prezo;
```

5 PAQUETES EN JAVA

Os paquetes en Java son un mecanismo para organizar e agrupar clases e outros tipos relacionados, como interfaces (que veremos máis adiante), de forma estruturada. En resume, proporcionan as seguintes vantaxes:

- ✓ Ofrecen un mecanismo para evitar conflitos de nomes.
- ✓ Melloran a modularidade.
- ✓ Facilitar a organización do código, e polo tanto o seu mantemento.

5.1 Funcións principais dos paquetes

Entre as principais funcións prácticas que ten a aplicación de paquetes en Java, destacan as seguintes:

- **Agrupación lóxica:** As clases relacionadas lóxicamente agrúpanse nun paquete para mellorar a organización do proxecto.
- **Evitar conflitos de nomes:** Grazas aos paquetes, diferentes clases poden ter o mesmo nome, sempre que pertencen a paquetes distintos.
- **Control de visibilidade:** Os paquetes permiten controlar a visibilidade (ou accesibilidade) das clases e membros, dentro do mesmo paquete ou entre diferentes. Isto verémolo no seguinte apartado onde se describen os modificadores de acceso.
- **Reutilizar código:** Facilitan que se poida reutilizar código ao importar paquetes específicos dentro dun proxecto, cando estes son necesarios.

5.2 Paquetes estándar

Java ofrece unha ampla biblioteca de paquetes estándar que conteñen clases e interfaces comúns para tarefas de programación. Algúns dos máis utilizados son:

- ➔ **java.lang:** Contén clases fundamentais da linguaxe Java, como poden ser: String, Math, Integer, etc. Este paquete impórtase automaticamente.
- ➔ **java.util:** Inclúe clases de utilidade, como ArrayList, HashMap, Random, etc.
- ➔ **java.io:** Proporciona clases para a entrada e saída, como File, InputStream, OutputStream, etc.
- ➔ **java.net:** Ofrece clases para comunicacións de rede como Socket, URL, etc.



5.3 Importar paquetes

Para usar clases ou interfaces dun paquete nunha clase, podemos importalas usando a palabra clave **import**.

Por exemplo:

```
import java.util.ArrayList;
```

Tamén podemos importar todos os recursos dun determinado paquete do seguinte xeito:

```
import java.util.*;
```

5.4 Crear paquetes

Para crear un paquete, engádese unha declaración de paquete ao principio (a primeira liña de código) do noso ficheiro Java.

Por exemplo:

```
package com.exemplo.proxecto;
```

Neste exemplo, créase un paquete chamado **com.exemplo.proxecto**.

5.5 Estrutura de directorios

Os paquetes seguen unha estrutura de directorios no sistema de ficheiros. Por exemplo, se tes o paquete **com.exemplo.proxecto**, a estrutura de directorios debería ser a seguinte:

```
com/  
  exemplo/  
    proxecto/  
      ClaseExemplo.java
```

Debemos ter en conta que se definimos un paquete nunha clase que non se atopa na estrutura de directorios que define, daranos erro ao compilar.

5.6 Paquetes aniñados (Sub-paquetes)

Java permite crear sub-paquetes dentro doutros, isto axuda a organizar mellor os compoñentes de grandes proxectos. Por exemplo, o paquete **java.util.concurrent** é un sub-paquete de **java.util**.



6 MODIFICADORES DE ACCESO

Os modificadores de acceso en Java son palabras clave que se utilizan para controlar o acceso aos membros (atributos e métodos) dunha clase. Estes permiten restrinxir ou permitir o acceso aos membros dunha clase desde outras clases.

A continuación temos unha descrición detallada dos principais modificadores de acceso en Java e cando é indicado o seu uso.

6.1 default (sen modificador)

Cando non se especifica ningún modificador, o acceso ao membro é "de paquete", o que significa que só pode ser accedido desde clases no mesmo paquete.

É útil cando queremos restrinxir o acceso a un membro da clase só para clases no mesmo paquete, mantendo a súa visibilidade limitada fóra dese paquete.

6.2 public (acceso total)

Os membros declarados como public poden ser accedidos desde calquera outra clase, xa sexa dentro do mesmo paquete ou doutro paquete. É útil cando queremos que os membros dunha clase sexan completamente accesibles dende calquera parte do programa.

Unha boa práctica é non asignar este tipo de modificador aos atributos, para que se accedan e modifiquen de xeito controlado a través de métodos (get e set).

6.3 protected (acceso limitado)

Os membros declarados como protected só poden ser accedidos desde clases no mesmo paquete ou dende clases fillo (subclases) desa clase.

É moi útil cando queremos restrinxir o acceso a un obxecto da clase mas queremos permitir que as subclases o accedan. Veremos a súa utilidade cando traballemos coa herdanza que ofrece a POO.

6.4 private (acceso restrinxido)

Os membros declarados como private só poden ser accedidos dende a propia clase na que se definen, de xeito que non se pode acceder a eles desde os obxectos ou instancias da clase. Trátase dun modificado moi útil cando queremos ocultar os detalles de implementación dunha clase, evitando así o acceso directo ou non controlado a determinados membros desde fóra da clase.



7 MÉTODOS

Un método en Java é un conxunto de declaracións que se utilizan para realizar unha acción específica, defínese dentro dunha clase e pode ser chamado para executar unha operación ou manipular datos. Un método pode ter ou non parámetros de entrada e pode devolver ou non un valor de retorno. A súa sintaxe é a seguinte:

```
[modificadores] tipoDeRetorno nomeDoMetodo(parámetros) {  
    // Corpo co código do método  
    [return valorDeRetorno]; // se procede devolver algo  
}
```

A continuación describimos os principais elementos dun método.

7.1 Estrutura dun método

7.1.1 Modificadores (opcional)

Os modificadores definen o comportamento e o acceso ao método. Algúns dos máis comúns son:

- **public**: O método pode ser chamado desde calquera parte do programa (de acceso).
- **private**: O método só pode ser chamado desde dentro da propia clase (de acceso).
- **protected**: O método pode ser chamado desde a clase ou subclases, e incluso en diferentes paquetes (de acceso).
- **static**: O método pertence á clase e non a unha instancia específica. Os métodos estáticos non poden acceder directamente aos membros non estáticos da clase.
- **final**: O método non pode ser sobrescrito polas subclases.
- **abstract**: O método non ten implementación na clase e debe ser implementado nas subclases (só aplicable en clases abstractas).
- **synchronized**: Só un fío pode executar o método á vez.

Poden ter máis dun modificador de distinto tipo, por exemplo:

```
public static void metodoA(String param1, Integer param2){  
    // Corpo do método  
}
```

7.1.2 Tipo de devolución

O tipo de devolución indica o tipo de dato que o método devolverá ao finalizar a súa execución. Pode ser calquera tipo de dato primitivo (como int, double, etc.) ou un obxecto dunha clase. Se o método non devolve ningún valor, o tipo de devolución será void.



Por exemplo:

```
public int sumar(int a, int b) {  
    return a + b; // Devolve un valor de tipo int  
}  
public void imprimirSaudo() {  
    System.out.println("Ola!");  
}
```

7.1.3 Nome

O nome do método debe ser único dentro da clase (os parámetros están incluídos no nome dun método). Segue as convencións de nomenclatura de Java, comezando con minúsculas e utilizando [camelCase](#).

7.1.4 Parámetros (opcional)

Os parámetros son as entradas que se lle poden pasar ao método para que execute operacións con elas. Podemos destacar o seguinte:

- Os parámetros van entre parénteses e especificase o tipo de dato seguido do nome da variable (tipo nomeParámetro).
- Poden existir varios parámetros, nese caso as súas definicións van separadas por comas.
- Son opcionais, se non se precisan parámetros, déixanse os parénteses baleiros.

Por exemplo:

```
public void imprimirNome(String nome) {  
    System.out.println("O nome é: " + nome);  
}  
public int multiplica(int a, int b) {  
    return a * b; // Devolve un valor de tipo int  
}
```

7.1.5 Corpo

O corpo contén declaracións e instrucións que executa o método cando se chama e que definen a lóxica do mesmo. Ten as seguintes características:

- Podes definir variables locais que só existen mentres o método se executa.
- Pode acceder sempre a todos os atributos e métodos da clase na que se define.
- Dentro deste bloque podemos realizar operacións e chamar a outros métodos aos que teñamos acceso.
- Dentro do corpo tamén podes usar estruturas de control como bucles ou condicións.



Por exemplo:

```
public int multiplicar(int x, int y) {  
    int resultado = x * y;  
    return resultado;  
}
```

7.1.6 Valor de Devolución (opcional)

Se o método devolve un valor, usaremos a palabra clave return seguida do valor que desexas devolver. Debemos ter en conta o seguinte:

- O tipo de dato do valor de retorno debe coincidir co [tipo de devolución declarado](#) no método.
- Se o método é void, non necesita devolver nada. Neste caso, se quixeramos forzar a finalización do método, poderíamos facer "return;" (sen indicar nada).

Por exemplo:

```
/**  
 * Método para calcular o prezo do coche despois de aplicar un desconto.  
 */  
 * @param desconto 0 desconto a aplicar (en porcentaxe).  
 * @return 0 prezo do coche despois de aplicar o desconto.  
 */  
public double calcularPrezoDescontado(double desconto) {  
    double prezoDescontado = prezo - (prezo * (desconto / 100));  
    return prezoDescontado;  
}  
  
/**  
 * Método para mostrar os detalles do coche.  
 */  
public void mostrarDetalles() {  
    System.out.println("Marca: " + marca);  
    System.out.println("Año de fabricación: " + anoFabricacion);  
    System.out.println("Prezo: " + prezo);  
}
```

7.2 Paso de parámetros: primitivos vs Obxectos

En Java, o paso de parámetros aos métodos segue o modelo de paso por valor tanto para tipos primitivos como para obxectos, pero o comportamento varía entre eles. A diferenza clave radica no seguinte:

- No caso dos tipos primitivos, pásase unha copia do valor.
- No caso dos obxectos, pásase unha copia da referencia ao obxecto, pero non a referencia



mesma.

7.2.1 Paso de parámetros con tipos primitivos

Cando pasamos como parámetro un valor primitivo (como int, double, etc.) a un método, pásase unha copia do valor que ten asignado nese momento. No caso de que o método modificara ese dato, como é unha copia, o valor orixinal fóra do método non se vería afectado.

Por exemplo:

```
/**
 * Clase que demostra o paso de parámetros con tipos primitivos.
 */
public class PasoPrimitivo {

    /**
     * Multiplica o número dado por dous.
     * @param numero valor primitivo (tipo int)
     */
    public void duplicarValor(int numero) {
        numero = numero * 2; // A variable local 'numero' modifícase, pero
        non afecta o valor orixinal
        System.out.println("Dentro do método, o número é: " + numero);
    }

    public static void main(String[] args) {
        int valor = 5;
        PasoPrimitivo exemplo = new PasoPrimitivo();

        System.out.println("Antes de chamar ao método, vale: " + valor);
        exemplo.duplicarValor(valor); // Pásase unha copia do valor
        System.out.println("Despois de excutar o método, segue sendo: " +
        valor); // O valor non cambia e segue a valer 5
    }
}
```

A saída da execución deste exemplo será:

```
Antes de chamar ao método, o valor é: 5
Dentro do método, o número é: 10
Despois de chamar ao método, o valor segue sendo: 5
```

Cando se chama a `duplicarValor(int numero)`, pásase unha copia de valor, polo que a modificación de “numero” dentro do método non afecta ao valor orixinal fóra do método.

7.2.2 Paso de parámetros con obxectos

Cando pasas un obxecto a un método, pásase unha copia da referencia ao obxecto, neste caso da dirección de memoria onde comeza a definición dese obxecto. Isto ten varias implicacións:



- Como temos unha referencia á posición de memoria dun obxecto, **todos os cambios que se fagan sobre o mesmo dentro do método, afectarán fora**. As dúas variables de tipo obxecto están apuntando á mesma posición de memoria, e polo tanto todo os cambios que se fan rexístranse sobre o mesmo espazo de memoria.
- O método non pode cambiar a referencia orixinal co operador "=", é dicir se fai iso, esa variable apuntaría a un novo obxecto e ese cambio non afectaría á variable orixinal.

Por exemplo:

```
/**
 * Clase que representa unha persoa con un nome.
 */
class Persoa {
    private String nome;

    // Construtor
    public Persoa(String nome) {
        this.nome = nome;
    }

    // Método getter para obter o nome
    public String getNome() {
        return nome;
    }

    // Método setter para cambiar o nome
    public void setNome(String nome) {
        this.nome = nome;
    }
}

public class ExemploPasoPorObxecto {

    /**
     * Modifica o atributo nome da persoa pasada como parámetro.
     * @param persoa un obxecto de tipo Persoa
     */
    public static void modificarNome(Persoa persoa) {
        persoa.setNome("Nome modificado"); // Modificamos o nome do
        obxecto orixinal
    }

    /**
     * Crea unha nova instancia de Persoa dentro do método.
     * @param persoa un obxecto de tipo Persoa
     */
    public static void asignarNovaReferencia(Persoa persoa) {
        persoa = new Persoa("Novo obxecto"); // Creamos un novo obxecto,
```



```
pero non afecta á referencia orixinal
}

public static void main(String[] args) {
    // Creamos un obxecto Persoa
    Persoa persoaOrixinal = new Persoa("Nome orixinal");

    // Mostramos o nome orixinal
    System.out.println("Nome antes de chamar a modificarNome: " +
persoaOrixinal.getNome());

    // Chamamos ao método modificarNome
    modificarNome(persoaOrixinal);
    System.out.println("Nome despois de chamar a modificarNome: " +
persoaOrixinal.getNome());

    // Chamamos ao método asignarNovaReferencia
    asignarNovaReferencia(persoaOrixinal);
    System.out.println("Nome despois de chamar a asignarNovaReferencia:
" + persoaOrixinal.getNome());
}
}
```

A saída do programa será a seguinte:

```
Nome antes de chamar a modificarNome: Nome orixinal
Nome despois de chamar a modificarNome: Nome modificado
Nome despois de chamar a asignarNovaReferencia: Nome modificado
```

A continuación imos facer unha pequena explicación:

1) Antes de modificar o obxecto:

- Creamos unha instancia de Persoa chamada persoaOrixinal con o nome "Nome orixinal".
- Mostramos o nome inicial, que é "Nome orixinal".

2) Modificar o nome dentro do método:

- Chamamos a modificarNome(persoaOrixinal). Este método cambia o atributo nome de persoaOrixinal a "Nome modificado".
- Como estamos modificando o obxecto ao que apunta a referencia, o cambio reflíctese fóra do método.
- O nome despois de chamar a este método é "Nome modificado".

3) Asignar unha nova referencia dentro do método:



- Chamamos a `asignarNovaReferencia(persoaOrixinal)`. Neste método, asignamos unha nova instancia de `Persoa` á variable local `persoa`, non á referencia orixinal.
- A nova referencia creada dentro do método non afecta á referencia orixinal, polo que cando imprimimos o nome despois desta chamada, segue sendo "Nome modificado".
- A nova referencia existe só dentro do método e non ten impacto fóra.

7.3 Construtores

Un construtor en Java é un método especial que se utiliza para inicializar obxectos dunha clase. A súa principal función é asignar valores iniciais aos atributos dun obxecto cando este se crea. Debemos entender que:

- ✓ É un método especial na clase que ten o mesmo nome que a clase.
- ✓ Non ten tipo de retorno explícito, nin sequer `void`.
- ✓ É invocado automaticamente cando se crea un novo obxecto da clase.
- ✓ Utilízase para inicializar os atributos do obxecto cun valor inicial.

A súa sintaxe básica é a seguinte:

```
public class NomeDaClase {  
    // Atributos da clase  
    // Constructor  
    public NomeDaClase(parámetros) {  
        // Inicialización dos atributos  
    }  
}
```

As características principais son:

- ➔ O nome do construtor debe ter o mesmo nome que a clase na que se declara.
- ➔ Pode ter parámetros que se utilizan para inicializar os atributos do obxecto. Estes parámetros son opcionais, dependendo dos requisitos da clase.
- ➔ Dentro do corpo do construtor, os atributos do obxecto poden ser inicializados utilizando os parámetros fornecidos, ou realizar calquera outra lóxica necesaria con eles

Exemplo:

```
/**  
 * Construtor que inicializa os atributos da instancia e incrementa o
```




```
* contador de coches cando se crea un novo
*
* @param marca          marca do coche
* @param anoFabricacion ano de fabricación
* @param prezo          prezo do coche
*/
public Coche(String marca, int anoFabricacion, double prezo) {
    this.marca = marca;
    this.anoFabricacion = anoFabricacion;
    this.prezo = prezo;
    contadorDeCoches++;
}
```

7.4 Métodos de acceso: Getters e Setters

Os métodos getters e setters son métodos públicos que se utilizan para acceder e modificar os valores de propiedades (atributos) privadas dunha clase en Java. O uso de encapsulamento, que é un dos principios fundamentais da Programación Orientada a Obxectos (POO), require ocultar os detalles internos das clases, limitando o acceso directo aos seus atributos.

1) Getters:

- Un getter é un método que permite ler o valor dun atributo privado.
- O seu nome adoita comezar por "get" seguido do nome do atributo con maiúscula inicial (notación [camelCase](#)).

2) Setters:

- Un setter é un método que permite modificar o valor dun atributo privado.
- O seu nome adoita comezar por "set" seguido do nome do atributo con maiúscula inicial.

A continuación temos un exemplo sinxelo disto:

```
public class Persoa {
    // Atributos privados
    private String nome;
    private int idade;

    // Constructor da clase
    public Persoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    /**
```



```
* Getter para obter o nome da persoa
* @return o nome da persoa
*/
public String getNome() {
    return nome;
}

/**
 * Setter para cambiar o nome da persoa
 * @param nome O novo nome da persoa
 */
public void setNome(String nome) {
    this.nome = nome;
}

/**
 * Getter para obter a idade da persoa
 * @return a idade da persoa
 */
public int getIdade() {
    return idade;
}

/**
 * Setter para cambiar a idade da persoa
 * @param idade A nova idade da persoa
 */
public void setIdade(int idade) {
    this.idade = idade;
}
}
```

Case todas as contornas de desenvolvemento, como [Eclipse](#) ou [IntelliJ IDEA](#) teñen opcións para xerar este tipo de métodos de xeito automático a partir dos atributos definidos na nosa clase. En Eclipse tan so temos que acceder a “Source → Generate Getters and Setters...”.

8 INSTANCIAR OBOJECTOS

Trátase do proceso de crear unha instancia (un obxecto) a partir dunha clase, pois como xa comentamos, unha clase en Java non é máis que un modelo ou molde que define as propiedades (atributos) e o comportamento (métodos) que terá o obxecto.

Para usar unha clase, é necesario crear un obxecto (ou instancia) a partir dela, polo que temos dous escenarios:

- Definir unha instancia a partir dunha clase propia, que temos que crear previamente.



- Definir unha instancia a partir de algunha clase dun recurso existente en Java (por exemplo, a clase String, Integer, etc).

No seguinte exemplo imos definir a clase Coche e despois crearemos unha instancia da mesma para traballar con ela.

Definir clase:

```
public class Coche {
    String marca;
    String modelo;
    int ano;

    // Constructor
    public Coche(String marca, String modelo, int ano) {
        this.marca = marca;
        this.modelo = modelo;
        this.ano = ano;
    }

    // Método para amosar a información do coche
    public void amosarInfo() {
        System.out.println("Marca: " + marca + ", Modelo: " + modelo + ",
Ano: " + ano);
    }
}
```

Crear instancia: Creamos unha clase App para poder probar o funcionamento da clase Coche, declaramos o método main e dentro declaramos un obxecto desa clase.

```
public class App {
    public static void main(String[] args) {
        Coche meuCoche = new Coche("Toyota", "Corolla", 2020); //
Instanciación dun obxecto
        meuCoche.amosarInfo(); // Chamando ao método da instancia
    }
}
```

9 COMPARAR OBXECTOS

En Java, a comparación de obxectos é un aspecto fundamental que require certa atención para evitar erros comúns, especialmente cando se tenta comparar obxectos complexos que non son tipos primitivos.

O operador == é adecuado para comparar referencias de obxectos, non os seus valores ou estados internos. Para comparar correctamente os contidos ou valores dun obxecto, hai que



seguir outras estratexias.

A continuación temos un exemplo do que indicamos:

```
// Creación de dúas cadeas con contido igual pero distintas instancias
String cadea1 = new String("Ola");
String cadea2 = new String("Ola");
// Creación dunha terceira cadea que apunta á mesma referencia de cadea1
String cadea3 = cadea1;

// Comparación con '=='
if (cadea1 == cadea2) {
    System.out.println("cadea1 e cadea2 apuntan á mesma referencia.");
} else {
    System.out.println("cadea1 e cadea2 NON apuntan á mesma referencia.");
}
// Comparación con '=='
if (cadea1 == cadea3) {
    System.out.println("cadea1 e cadea3 apuntan á mesma referencia.");
} else {
    System.out.println("cadea1 e cadea3 NON apuntan á mesma referencia.");
}
```

A saída dese código será:

```
cadea1 e cadea2 NON apuntan á mesma referencia.
cadea1 e cadea3 apuntan á mesma referencia.
```

No exemplo anterior temos que:

- **cadea1 e cadea2 son creadas usando new String("Ola"):** polo que, a pesar de que teñen o mesmo contido, son dúas instancias distintas, cada unha ocupando un lugar diferente na memoria. O operador == entre elas devolve false, xa que comparan referencias e non o contido.
- **cadea3 recibe a referencia de cadea1:** O operador == entre cadea1 e cadea3 devolve true, porque ambas variables apuntan ao mesmo obxecto en memoria.

Para comparar correctamente esas dúas cadeas deberíamos usar o método equals que teñen todos os obxectos, pois está definido na superclase Object, e nas vindeiras unidades veremos como se pode adaptar para cada clase. Partindo do exemplo anterior, teríamos:

```
// Creación de dúas cadeas con contido igual pero distintas instancias
String cadea1 = new String("Ola");
String cadea2 = new String("Ola");
```



```
// Creación dunha terceira cadea que apunta á mesma referencia de cadea1
String cadea3 = cadea1;

// Comparación correcta de cadea1 e cadea2 usando equals()
if (cadea1.equals(cadea2)) {
    System.out.println("cadea1 e cadea2 teñen o mesmo contido.");
} else {
    System.out.println("cadea1 e cadea2 NON teñen o mesmo contido.");
}

// Comparación correcta de cadea1 e cadea3 usando equals()
if (cadea1.equals(cadea3)) {
    System.out.println("cadea1 e cadea3 teñen o mesmo contido.");
} else {
    System.out.println("cadea1 e cadea3 NON teñen o mesmo contido.");
}
```

A saída dese código será:

```
cadea1 e cadea2 teñen o mesmo contido.
cadea1 e cadea3 teñen o mesmo contido.
```

No exemplo anterior temos que:

- **cadea1 e cadea2** son dúas instancias diferentes que conteñen o mesmo valor de texto ("Ola"). Ao usar `equals()`, Java compara os caracteres almacenados nelas e devolve `true` se son iguais.
- **cadea3** apunta ao mesmo obxecto que `cadea1`, polo que tamén teñen o mesmo contido

10 CLASES WRAPPER

Até o de agora traballamos cos tipos de datos primitivos, mais a linguaxe Java ten para cada un deles unha clase wrapper (envolvente) correspondente. Estas permiten tratar tipos de datos primitivos como obxectos, o que é moi útil cando se necesita traballar en contextos que requiren obxectos, como as coleccións de Java (`ArrayList`, `HashMap`, etc.), que non poden almacenar tipos primitivos directamente.



Na seguinte táboa podemos ver os tipos de datos primitivos cos seus wrappers correspondentes:

Tipo primitivo	Clase wrapper	Tamaño (bits)	Valor por defecto (primitivo)	Valor por defecto (wrapper)
int	Integer	32	0	null
float	Float	32	0.0f	null
double	Double	64	0.0d	null
boolean	Boolean	1	false	null
char	Character	16	'\u0000' (espazo en branco)	null
byte	Byte	8	0	null
short	Short	16	0	null
long	Long	64	0L	null

10.1 Diferenzas cos tipos primitivos

Existen unha serie de diferenzas entre as clases wrapper e os tipos primitivos:

- **Natureza:**
 - Os tipos primitivos son tipos de datos básicos que almacenan directamente valores. Non son obxectos.
 - As clases wrapper son obxectos que envolver un valor primitivo e proporcionan métodos útiles para manipular, converter, ou comparar valores.
- **Uso de memoria:**
 - Os primitivos son máis lixeiros en termos de uso de memoria e almacenan directamente os seus valores.
 - As clases wrapper consumen máis memoria xa que son obxectos e inclúen metadatos e métodos, ademais do valor en si.
- **Valores por defecto:**
 - Para os tipos primitivos, se unha variable non se inicializa explicitamente, asignarase



un valor por defecto, como 0 para números ou false para booleanos.

- As clases wrapper permiten que os valores sexan null, o que indica que non se asignou ningún valor ao obxecto (tal como acontece no resto de obxectos).

10.2 Autoboxing e Unboxing

A partir de Java 5, o compilador de Java pode converter automaticamente entre tipos primitivos e as súas clases wrapper correspondentes. Isto ten dúas denominacións:

- **Autoboxing**: cando un primitivo se converte nun obxecto wrapper.

```
int primitivo = 10;  
Integer wrapper = primitivo;  
// Autoboxing: convértese automaticamente nun Integer
```

- **Unboxing**: cando un obxecto wrapper se converte no seu tipo primitivo.

```
Integer wrapper = 15;  
int primitivo = wrapper;  
// Unboxing: o Integer convértese automaticamente en int
```

10.3 Principais métodos

As clases wrapper proporcionan diversos métodos útiles que os tipos primitivos non teñen. Abaixo hai algúns exemplos de métodos comúns dispoñibles nas clases wrapper que facilitan o traballo con estes tipos:

- **Métodos das clases Integer, Float, Double**:
 - **parseInt(String s), parseFloat(String s), parseDouble(String s)**: Converte unha cadea nun valor numérico.
 - **toString()**: Converte o valor nunha cadea de texto.
 - **valueOf(String s)**: Devolve unha instancia da clase wrapper correspondente, convertendo unha cadea ao tipo apropiado.
 - **compareTo()**: Compara dous valores e devolve -1, 0, ou 1, dependendo de se o valor é menor, igual, ou maior que o outro.

Exemplo con Integer:

```
String numeroComoCadea = "123";  
int num = Integer.parseInt(numeroComoCadea); // Converte a cadea nun  
int  
Integer outroNum = Integer.valueOf(numeroComoCadea); // Converte a  
cadea nun Integer
```



- **Métodos da clase Boolean:**

- **parseBoolean(String s):** Converte unha cadea de texto ("true" ou "false") nun valor booleano.
- **valueOf(String s):** Converte unha cadea de texto nun obxecto Boolean.

Exemplo con Boolean:

```
String verdade = "true";  
// Converte a cadea nun boolean  
boolean bool = Boolean.parseBoolean(verdade);  
  
// Converte a cadea nun Boolean  
Boolean boolWrapper = Boolean.valueOf(verdade);
```

10.4 Diferenzas ao comparar tipos primitivos e wrappers

Ao comparar tipos primitivos e os seus wrappers, hai que ter coidado, especialmente cando se trata de null e ao comparar obxectos en lugar de valores. Por exemplo:

- **Comparación con primitivos:** Os tipos primitivos compáranse directamente utilizando o operador ==.

```
int a = 5;  
int b = 5;  
System.out.println(a == b); // true
```

- **Comparación con wrappers:** Cando se comparan wrappers, non debes utilizar ==, xa que isto compara a referencia do obxecto, non o valor. Para comparar valores de obxectos wrapper, debes usar o método equals().

```
Integer x = new Integer(5);  
Integer y = new Integer(5);  
  
System.out.println(x == y); // false, compara referencias  
System.out.println(x.equals(y)); // true, compara valores
```

11 CLASES DESTACADAS EN JAVA

En Java, ademais de poder definir e declarar as nosas propias clases e recursos, tamén temos numerosas bibliotecas que facilitan o traballo do programador. Este mecanismo para poder reutilizar código e recursos é o que fai realmente potente o paradigma da Programación Orientada a Obxectos (POO). A continuación describimos algunhas das máis utilizadas.



11.1 Superclase Object

A clase `Java.lang.Object` é a superclase de todas as demais en Java, pois todos os obxectos en Java herdan, directa ou indirectamente, desta clase (aínda que non o indiquen de xeito explícito). Algunhas dos métodos máis importantes que ten, e que polo tanto teñen todas as clases de xeito nativo, son as seguintes:

- **`toString()`**: Devólvelle unha representación en forma de cadea do obxecto.
- **`equals()`**: Compara se dous obxectos son iguais.
- **`hashCode()`**: Devólvelle un código hash do obxecto.
- **`clone()`**: Crea e devolve unha copia do obxecto.

Na seguinte unidade veremos o que é a herdanza, e como podemos crear as nosas clases herdadas, e modificar e adaptar métodos herdados como estes que se describiron.

11.2 Clase Math

A clase `java.lang.Math` contén métodos estáticos que permiten realizar operacións matemáticas básicas e avanzadas, como logaritmos, potencias, trigonometría, e arredondamento. Algúns dos métodos máis importantes son:

→ Métodos de arredondamento:

- ✓ **`Math.round(double a)`**: Arredonda ao enteiro máis próximo.
- ✓ **`Math.ceil(double a)`**: Arredonda para arriba ao enteiro máis próximo.
- ✓ **`Math.floor(double a)`**: Arredonda para abaixo ao enteiro máis próximo.

→ Operacións de potencias e raíces:

- ✓ **`Math.pow(double a, double b)`**: Calcula a elevado á potencia de b.
- ✓ **`Math.sqrt(double a)`**: Devolve a raíz cadrada de a.

→ Operacións trigonométricas:

- ✓ **`Math.sin(double a)`, `Math.cos(double a)`, `Math.tan(double a)`**: Calcula o seno, o coseno, ou a tanxente de a (en radiáns).
- ✓ **`Math.toRadians(double degrees)`**: Convértese de graos a radiáns.
- ✓ **`Math.toDegrees(double radians)`**: Convértese de radiáns a graos.



→ Outros métodos:

- ✓ **Math.abs(int a):** Devolve o valor absoluto de a.
- ✓ **Math.max(int a, int b), Math.min(int a, int b):** Calcula o valor máximo ou mínimo entre a e b.
- ✓ **Math.random():** Devolve un número aleatorio entre 0.0 e 1.0 (maior que 0 e menor que 1).

Exemplo de uso:

```
double resultado = Math.pow(2, 3); // Resultado: 8.0
double raiz = Math.sqrt(25); // Resultado: 5.0
double maximo = Math.max(10, 20); // Resultado: 20
```

11.3 Clase Random

A clase Random úsase para xerar números aleatorios en Java. Pode xerar números enteiros, números de punto flotante, e booleanos. Está incluída no paquete java.util e é amplamente usada en simulacións, xogos, probas, e calquera situación onde sexa necesario xerar datos aleatorios.

11.3.1 Características principais

Os números xerados por esta clase non son verdadeiramente aleatorios, senón que son pseudoaleatorios, é dicir, seguen un algoritmo determinista que xera unha secuencia de números aparentemente aleatorios baseados nun valor inicial chamado seed (semente).

Se non se especifica un valor seed, entón a clase usa o tempo do sistema para inicializalo. No caso de especificalo, entón a secuencia de números será reproducible, é dicir, cada vez que se use o mesmo seed, obtéñense os mesmos resultados.

Por exemplo, imos crear un obxecto random indicando o seed e outro sen indicalo:

```
Random random1 = new Random();
Random random2 = new Random(42);
```

11.3.2 Métodos para xerar números por tipo

Random conta con métodos que permiten xerar números aleatorios de diferentes tipos:

- **int nextInt():** Devolve un número enteiro aleatorio en todo o rango posible para int, desde Integer.MIN_VALUE ata Integer.MAX_VALUE.

```
int aleatorio = random.nextInt();
```

- **int nextInt(int bound):** Devolve un número enteiro aleatorio entre 0 (inclusivo)



e o valor bound especificado (exclusivo).

```
int aleatorio = random.nextInt(10); // Xera un número entre 0 e 9
```

- **float nextFloat():** Devolve un número en coma flotante aleatorio entre 0.0 (inclusivo) e 1.0 (exclusivo).

```
float aleatorio = random.nextFloat();
```

- **double nextDouble():** Devolve un número en coma flotante de dobre precisión entre 0.0 (inclusivo) e 1.0 (exclusivo).

```
double aleatorio = random.nextDouble();
```

- **long nextLong():** Devolve un número enteiro longo aleatorio, desde Long.MIN_VALUE ata Long.MAX_VALUE.

```
long aleatorio = random.nextLong();
```

- **boolean nextBoolean():** Devolve un valor booleano aleatorio, true ou false.

```
boolean aleatorio = random.nextBoolean();
```

11.3.3 Outras consideracións

A continuación detallamos algunhas consideracións sobre o uso da clase Random:

- **Reproducibilidade:** Se necesitas que o comportamento aleatorio sexa reproducible (por exemplo, para probas), sempre podes usar un valor seed fixo.
- **Thread-safety:** A clase Random non é segura para fíos (non é thread-safe). Se precisas usar números aleatorios en aplicacións que empreguen múltiples fíos, é recomendable usar `java.util.concurrent.ThreadLocalRandom`, que está deseñada especificamente para ser segura e eficiente en contornas onde hai múltiples fíos de execución.
- **Outras alternativas:** Para unha maior seguridade en aplicacións que requiren números aleatorios criptograficamente seguros, podes utilizar `java.security.SecureRandom`, que está deseñada para ofrecer números aleatorios máis seguros a nivel criptográfico.

11.3.4 Exemplos

11.3.4.1 Simulación dun dado de 6 caras

Imaxínade que queremos simular un dado de 6 caras (números do 1 ao 6). No seguinte exemplo, utilizamos `nextInt(6)` para xerar un número entre 0 e 5, e despois sumamos 1



para ter un número entre 1 e 6.

```
Random random = new Random();  
int resultadoDado = random.nextInt(6) + 1; // Xera un número entre 1 e 6  
System.out.println("Resultado do dado: " + resultadoDado);
```

11.3.4.2 Xerar un número real entre dous límites

Para xerar un número en coma flotante entre dous límites (por exemplo, entre min e max), podemos facer o seguinte:

```
double min = 10.0;  
double max = 20.0;  
double aleatorio = min + (max - min) * random.nextDouble(); // Entre 10.0  
e 20.0  
System.out.println("Número aleatorio: " + aleatorio);
```

11.3.4.3 Xerar un número real entre dous límites

Para xerar un número en coma flotante entre dous límites (por exemplo, entre min e max), podemos facer o seguinte:

```
double min = 10.0;  
double max = 20.0;  
double aleatorio = min + (max - min) * random.nextDouble(); // Entre 10.0  
e 20.0  
System.out.println("Número aleatorio: " + aleatorio);
```

11.3.4.4 Simulación dun sorteo aleatorio

Nun sorteo de números entre 1 e 100, podemos seleccionar un número ao azar deste rango:

```
Random random = new Random();  
int sorteado = random.nextInt(100) + 1; // Número entre 1 e 100  
System.out.println("Número sorteado: " + sorteado);
```

11.4 Clase String

A clase String é probablemente unha das máis utilizadas en Java, xa que representa cadeas de texto. Debemos ter en conta que a clase String é immutable, o que significa que unha vez creada unha cadea, o seu valor non pode ser modificado. Calquera operación que altere unha cadea devolve unha nova instancia de String, sen modificar o obxecto orixinal.

r



11.4.1 Características principais

Para traballar con esta clase, temos dous xeitos de crear as instancias ou obxectos da mesma:

- **Usando literais de cadea:** esta é a forma recomendada, xa que é máis eficiente:

```
String str = "Ola mundo"; // Creación dunha cadea cun literal.  
String outraStr = new String("Texto"); // Creación co construtor
```

- **Usando o operador new:** Tamén podemos crear unha nova instancia co mesmo operador que usamos para o resto de clases:

```
String cadea = new String("Ola Mundo!");
```

11.4.2 Principais métodos

A clase String ofrece unha ampla variedade de métodos para manipular cadeas de texto.

A continuación, vemos algúns dos máis comúns e útiles:

- ✓ **int length():** Devolve a lonxitude da cadea.

```
String cadea = "Ola";  
int lonxitude = cadea.length(); // lonxitude = 3
```

- ✓ **charAt(int index):** Devolve o carácter na posición indicada.

```
String cadea = "Ola";  
char primeiroCaractere = cadea.charAt(0); // primeiroCaractere = 'O'
```

- ✓ **toLowerCase():** Devolve unha copia da cadea en minúsculas.

```
String cadea = "Ola Mundo";  
String minusculas = cadea.toLowerCase(); // minusculas = "ola mundo"
```

- ✓ **toUpperCase():** Devolve unha copia da cadea en maiúsculas.

```
String cadea = "Ola Mundo";  
String maiusculas = cadea.toUpperCase(); // maiusculas = "OLA MUNDO"
```

- ✓ **substring(int beginIndex, int endIndex):** Devolve a subcadea que comeza na posición beginIndex ata o final da cadea.

```
String cadea = "Ola Mundo";  
String subcadea = cadea.substring(4); // subcadea = "Mundo"
```

- ✓ **String substring(int beginIndex, int endIndex):** Devolve a subcadea que comeza en



beginIndex e remata en endIndex (exclusivo).

```
String cadea = "Ola Mundo";  
String subcadea = cadea.substring(0, 3); // subcadea = "Ola"
```

- ✓ **indexOf(String str):** Devolve a primeira posición onde se atopa a sub-cadea especificada.

```
String cadea = "Ola Mundo";  
int posicion = cadea.indexOf("Mundo"); // posicion = 4
```

- ✓ **int lastIndexOf(String str):** Devolve a posición da última aparición dunha subcadea, ou -1 se non se atopa.

```
String cadea = "Ola Mundo Ola";  
int posicion = cadea.lastIndexOf("Ola"); // posicion = 10
```

- ✓ **trim():** Devolve unha copia da cadea cos espazos en branco ao comezo e ao final eliminados.

```
String cadea = " Ola Mundo ";  
String senEspazos = cadea.trim(); // senEspazos = "Ola Mundo"
```

- ✓ **boolean contains(CharSequence s):** Devolve true se a cadea contén a secuencia de caracteres especificada.

```
String cadea = "Ola Mundo";  
boolean contén = cadea.contains("Mundo"); // contén = true
```

- ✓ **boolean equals(String anotherString):** Compara se dúas cadeas son iguais (diferencia maiúsculas de minúsculas).

```
String cadea1 = "Ola";  
String cadea2 = "Ola";  
boolean sonIguais = cadea1.equals(cadea2); // sonIguais = true
```

- ✓ **boolean equalsIgnoreCase(String anotherString):** Compara cadeas sen diferenciar entre maiúsculas e minúsculas.

```
String cadea1 = "ola";  
String cadea2 = "Ola";  
boolean sonIguais = cadea1.equalsIgnoreCase(cadea2);  
// sonIguais = true
```

- ✓ **split(String regex):** Devolve unha copia da cadea dividida segundo un patrón indicado na



expresión regular.

```
String cadea = "Ola, Mundo, Java";  
String[] partes = cadea.split(", ");  
// partes = {"Ola", "Mundo", "Java"}
```

- ✓ **boolean isEmpty():** Devolve true se a cadea está baleira (non contén caracteres).

```
String cadea = "";  
boolean estaBaleira = cadea.isEmpty(); // estaBaleira = true
```

- ✓ **boolean isBlank()** : Devolve true se a cadea está baleira ou contén só espazos en branco (a partires de Java v11).

```
String cadea = " ";  
boolean estaBaleiraOuEnBranco = cadea.isBlank();  
// estaBaleiraOuEnBranco = true
```

Exemplo de uso:

```
String texto = "Ola mundo";  
int lonxitude = texto.length(); // 9  
String subcadea = texto.substring(0, 3); // "Ola"  
boolean iguais = texto.equals("ola mundo"); // false  
String enMaiusculas = texto.toUpperCase(); // "OLA MUNDO"
```

12 EXEMPLOS

12.1 Xogo de adiviña a palabra

No seguinte exemplo temos un xogo sinxelo de adiviñar palabras en Java, que utiliza a clase String e varios dos seus métodos. O obxectivo é que o usuario adiviñe unha palabra descoñecida, proporcionándolle pistas a medida que avanza a partida. O código está documentado usando o estándar Javadoc para que sexa didáctico e fácil de seguir.

```
/**  
 * A clase XogoAdivinhaPalabra simula un xogo sinxelo onde o usuario debe  
 * adiviñar unha palabra oculta  
 * baseándose nas pistas proporcionadas polo programa.  
 *  
 * Usa varios métodos da clase String para manipular e comparar cadeas.  
 *  
 * @author Manuel Pacior Pérez  
 * @version 1.0  
 */  
public class XogoAdivinhaPalabra {
```



```
/**
 * Método principal que executa o xogo de adiviñar a palabra.
 *
 * @param args Non se utilizan parámetros nesta aplicación
 */
public static void main(String[] args) {
    // Palabra que o usuario debe adiviñar
    String palabraOculta = "dam";

    // Número máximo de intentos
    int maxIntentos = 5;
    int intentos = 0;
    boolean adiviñada = false;

    // Scanner para ler as entradas do usuario
    Scanner scanner = new Scanner(System.in);

    System.out.println("Benvido ao xogo de adiviñar a palabra!");
    System.out.println("Tes " + maxIntentos + " intentos para adiviñar a palabra oculta.");

    // Bucle principal do xogo
    while (intentos < maxIntentos && !adiviñada) {
        System.out.println("\nPista: " + obterPista(palabraOculta, intentos));

        System.out.print("Introduce a túa adiviñanza: ");
        String adiviñanza = scanner.nextLine();

        // Comparar a adiviñanza co contido da palabra oculta (insensible a maiúsculas)
        if (compararCadeas(palabraOculta, adiviñanza)) {
            adiviñada = true;
            System.out.println("Parabéns! Adiviñaches a palabra correcta!");
        }
        else {
            intentos++;
            System.out.println("Non é correcto. Quédanche " + (maxIntentos - intentos) + " intentos.");
        }
    }

    // Se non adiviñou a palabra
    if (!adiviñada) {
        System.out.println("Oh! Quedácheste sen intentos. A palabra era: " + palabraOculta);
    }

    scanner.close();
}
```




```
/**
 * Compara dúas cadeas de forma insensible a maiúsculas/minúsculas.
 *
 * @param palabraOculta A palabra que o usuario debe adiviñar
 * @param adiviñanza A palabra que introduce o usuario
 * @return true se as dúas cadeas son iguais en contido,
 ignorando maiúsculas/minúsculas
 */
public static boolean compararCadeas(String palabraOculta, String
adiviñanza) {
    return palabraOculta.equalsIgnoreCase(adiviñanza);
}

/**
 * Obtén unha pista para mostrar ao usuario as primeiras letras da
 palabra oculta. A medida que aumentan os
 * intentos, a pista mostra máis letras da palabra.
 *
 * @param palabra A palabra oculta
 * @param intentos O número de intentos xa realizados polo usuario
 * @return Unha cadea que contén parte da palabra oculta como
 pista
 */
public static String obterPista(String palabra, int intentos) {
    // Calcula a lonxitude da pista en función dos intentos (1 letra
 extra por intento)
    int lonxitudePista = intentos + 1;
    if (lonxitudePista > palabra.length()) {
        lonxitudePista = palabra.length(); // Non excede a lonxitude
 da palabra
    }
    // Retorna o substring que corresponde á pista e engade asteriscos
 para as letras ocultas
    String pista = palabra.substring(0, lonxitudePista);
    for (int i = lonxitudePista; i < palabra.length(); i++) {
        pista += "*"; // Oculta as letras restantes con asteriscos
    }
    return pista;
}
}
```

12.2 Clase Coche

No seguinte exemplo temos unha clase Coche con variables de instancia *marca*, *anoFabricacion* e *prezo*. Temos un construtor para inicializar estas variables e un método *mostrarDetalles()* para mostrar información do coche.

```
/**
 * Clase que representa un coche e conta o número total de coches creados.
```



```
*/
public class Coche {

    /** Marca do coche. Atributo final */
    private final String marca;

    /** Ano de fabricación do coche. Atributo final */
    private final int anoFabricacion;

    /** Prezo do coche. Atributo final */
    private final double prezo;

    /** Km percorridos polo coche. */
    private double km = 0;

    /** Contador de coches. */
    private static int contadorDeCoches;

    /**
     * Construtor que inicializa os atributos da instancia e incrementa o
     * contador de coches cando se crea un novo.
     *
     * @param marca          marca do coche
     * @param anoFabricacion ano de fabricación
     * @param prezo          prezo do coche
     */
    public Coche(String marca, int anoFabricacion, double prezo) {
        this.marca = marca;
        this.anoFabricacion = anoFabricacion;
        this.prezo = prezo;
        contadorDeCoches++;
    }

    /**
     * Método para mostrar os detalles do coche.
     */
    public void mostrarDetalles() {
        System.out.println("Marca: " + marca);
        System.out.println("Ano de fabricación: " + anoFabricacion);
        System.out.println("Prezo: " + prezo);
    }

    /**
     * Método para calcular o prezo do coche despois de aplicar un desconto.
     *
     * @param desconto O desconto a aplicar (en porcentaxe).
     * @return O prezo do coche despois de aplicar o desconto.
     */
    public double calcularPrezoDescontado(double desconto) {
        double prezoDescontado = prezo - (prezo * (desconto / 100));
        return prezoDescontado;
    }
}
```



```
}

/**
 * Obtén os km do coche
 *
 * @return
 */
public double getKm() {
    return km;
}

/**
 * Configura os Km do coche
 *
 * @param km km do coche
 */
public void setKm(double km) {
    this.km = km;
}

/**
 * Método para obter a marca do coche.
 *
 * @return A marca do coche.
 */
public String getMarca() {
    return marca;
}

/**
 * Método para obter o ano de fabricación do coche.
 *
 * @return O ano de fabricación do coche.
 */
public int getAnoFabricacion() {
    return anoFabricacion;
}

/**
 * Método para obter o prezo do coche.
 *
 * @return O prezo do coche.
 */
public double getPrezo() {
    return prezo;
}

/**
 * Obtén o número total de obxectos de tipo Coche creados
 *
 * @return the contadorDeCoches
```



```
*/  
public static int getContadorDeCoches() {  
    return contadorDeCoches;  
}  
  
}
```

Para probar ese código poderíamos ter unha clase como esta:

```
/**  
 * Clase que demostra o uso da clase Coche.  
 */  
public class Principal {  
    /**  
     * Método principal para probar a clase Coche.  
     *  
     * @param args Argumentos da liña de comandos (non utilizado).  
     */  
    public static void main(String[] args) {  
        // Crear obxectos Coche  
        Coche coche1 = new Coche("Toyota", 2019, 15000.0);  
        Coche coche2 = new Coche("Ford", 2017, 12000.0);  
  
        // Mostrar detalles dos coches  
        System.out.println("Detalles do coche 1:");  
        coche1.mostrarDetalles();  
  
        System.out.println("\nDetalles do coche 2:");  
        coche2.mostrarDetalles();  
    }  
}
```

Explicación:

- ➔ **private:** Os atributos marca, anoFabricacion e prezo son privados, o que significa que só poden ser accedidos directamente dentro da propia clase Coche. Isto axuda a encapsular os detalles internos da clase e prevén que sexan modificados de maneira inadecuada desde fóra da clase.
- ➔ **Public:** Os métodos getMarca(), setMarca(), getPrezo(), setPrezo() e getAnoFabricacion() e setAnoFabricacion() son públicos, o que significa que poden ser accedidos desde calquera parte do programa. Estes métodos permiten acceder e modificar os atributos privados de Coche de forma controlada, sen permitir un acceso directo a eles.

Estes modificadores de acceso axudan a asegurar que os atributos da clase sexan accedidos e modificados de maneira segura e controlada, seguindo o principio de encapsulación.